

Important Notes Of MySQL Part 3

*

Software Corporate Trainer

:Chandrasekhar N

Foreign Key

Foreign key is a key which is a primary key in another table.

SQL> desc salespeople;

Name

Null?

Type

SNUM

	NOT NULL NUMBER(5)
SNAME	
	CHAR(25)
CITY	
	VARCHAR2(20)
COMM	
	NUMBER(12,2)

Creating customer table with
 snum as foreign key connecting to
 parent table
 salespeople;

```
mySQL> create table customers
(cnum numeric(5) primary key,
  cname char(28),
  city varchar(20),
  snum numeric(5) references
salespeople(snum));
```

Table created.

Creating Orders table with snum
and cnum as foreing key
connecting to respected parent
tables
salespeople and customers.

```
mySQL> create table orders
        (onum numeric(5) primary
key,
        odate date,
        oamount numeric(11,2),
        snum numeric(5) references
salespeople(snum),
        cnum numeric(5) references
customers(cnum));
```

Table created.

SQL - LIKE Clause

The SQL LIKE clause is used to compare a value to similar values using wildcard operators.

There are two wildcards used in conjunction with the LIKE operator:

The percent sign (%)

The underscore (_)

The percent sign represents zero, one, or multiple characters.

The underscore represents a single number or character.

The symbols can be used in

combinations.

```
*****  
*****  
*****  
**
```

```
mysql>SELECT * FROM salespeople  
        WHERE empname like 'D%';
```

The above query will display
whose names begins with D
% sign represent any characters
but the first character must
begins with character D.

```
-----  
-----  
-----
```

waq where u will display
salespeople whose city name
begins
with A

```
sql>SELECT * FROM salespeople
```

```
WHERE city like 'A%';
```

```
-----  
-----  
-----
```

w.a.q when you will print the
sname, city, comm for all people
residing in
London(Use like operator)

```
mysql>Select snum, sname, city  
FROM salespeople  
WHERE city like 'L%';
```

Like operator is not case
sensitive.

```
*****
```

```
mysql>  
Select * from salespeople  
WHERE comm LIKE '200%'
```

The above query will display
those salespeople whose salary

attribute

first 3 digist are 200

_ (underscore) in like operator
represent 1 character or number
or space or special
symbol.

sql>

```
select * from salespeople
where sname like '_____';
```

In the above query we will
display only those names which
are of 5 characters.

one _ underscore represent one
character.

```
sql> select * from salespeople
where city like '_____';
```

The above query will print city whose name size is of 6 characters.

```
sql>select * from customers  
where cname like 'a%';
```

The above query will display all those cnames that begins with a

```
=====
```

```
=====
```

```
=====
```

SQL - Distinct Keyword

The SQL DISTINCT keyword is used in conjunction with SELECT statement to eliminate all the duplicate records and fetching only unique records.

There may be a situation when you

have multiple duplicate records in a table. While fetching such records, it makes more sense to fetch only unique records instead of fetching duplicate records.

Syntax:

The basic syntax of DISTINCT keyword to eliminate duplicate records is as follows:

```
sql>select distinct city from  
salespeople;
```

```
sql> select count(distinct city)  
from salespeople;
```

```
=====
```

```
=====
```

```
=====
```

Between Operator

The BETWEEN operator is used to select values within a range.

The SQL BETWEEN Operator

The BETWEEN operator selects values within a range. The values can be numbers, text, or dates.

```
sql> SELECT * FROM salespeople
      WHERE comm between 10000 and
      20000;
```

The following SQL statement selects all salespeople whose commission is between 10000 and 20000

```
=====
=====
=====
```

NOT BETWEEN Operator Example

```
-----
```

To display the employees outside the range of the previous example, use NOT BETWEEN:

Example

```
sql> SELECT * FROM customers
      WHERE comm NOT BETWEEN 10000
AND 20000;
```

The above query will print only those salespeople whose salary does not fall in the above range.

BETWEEN Operator with Text Value Example

The following SQL statement selects all salespeople whose

name begins
with any of the letter BETWEEN
'C' and 'M':

```
sql> SELECT * FROM salespeople;  
      WHERE Name BETWEEN 'C' AND  
'M';
```


BETWEEN Operator with IN Example

The following SQL statement
selects all employees with a
salary

BETWEEN 50000 AND 150000

but employee number of 99, 2 or
390 should not be displayed:

```
sql> SELECT * FROM employee  
      WHERE (basic BETWEEN 50000  
AND 150000)
```

```
        AND NOT empno IN
(99,2,390,66);
```

In the above query we are using the a sub query and also the in operator with not operator so that only thos employees are print who draw salary in the given range but particular employees no should not be printed due to what ever reason as the client wants.

NOT BETWEEN Operator with Text Value Example

The following SQL statement selects all employees with empno

beginning with any of the letter
NOT BETWEEN 'C' and 'M':

```
sql>SELECT * FROM employee
      WHERE Name not BETWEEN 'C'
AND 'M';
```

```
=====
=====
=====
```

Assume for the following examples
we have an employee table.

```
*****
*****
```

ORDER BY Clause

```
*****
```

The SQL ORDER BY clause is used
to sort the data in ascending or
descending order, based on one or
more columns.

You can use more than one column in the ORDER BY clause.

Make sure whatever column you are using to sort, that column should be in column-list.

waq where you will sort on employee name sorting based on salary in ascending order.

```
SQL> SELECT * FROM employee  
      ORDER By SALARY;
```

```
-----  
-----
```

Following is the query where we sort only by name in ascending order.

```
SQL> SELECT * FROM EMPLOYEE  
      order by empname;
```

```
sql> select empno, city, basic  
from employee
```

```
order by city;
```

in the above query the records are sorted based on city in ascending order.

```
-----  
-----
```

Following is an example which would sort the result in descending order by city:

```
SQL> SELECT * FROM employee  
      ORDER BY city DESC;
```

or

```
sql> select empname, city from  
employee  
      order by city desc;
```

```
*****  
*****
```

```
SQL> SELECT * FROM employee
```

ORDER BY NAME;

The above query will sort the record based on name in ascending order.


```
sql> select empname, basic from  
employee  
       order by basic desc;
```

the above query will sort in descending order.

```
*****  
*****  
*****  
*****
```

Group By

```
*****
```

The SQL GROUP BY clause is used in collaboration with the SELECT statement to arrange identical

data into groups.

The GROUP BY clause follows the WHERE clause in a SELECT statement and precedes the ORDER BY clause.

Syntax:

The basic syntax of GROUP BY clause is given below.

The GROUP BY clause must follow the conditions in the WHERE clause and must precede the ORDER BY clause if one is used.

```
SELECT column1, column2  
FROM table_name  
WHERE [ conditions ]  
GROUP BY column1, column2  
ORDER BY column1, column2
```

If you want to know the total amount of salary on each designation,

then GROUP BY query would be as follows:

```
SQL> SELECT desig, SUM(basic)
FROM employee
      GROUP BY desig;
```

IN the above query group by is used so that the query in the memory of the computer will group the tuples based on designation and then each group will have only 1 output in the screen along with the salary total of each group.

write a query where you will print the maximum salary drawn in each group using aggregate functions.

```
sql> SELECT desig, Max(basic)
FROM employee
```

GROUP BY desig;

sql> SELECT city SUM(basic) FROM
employee

GROUP BY city;

in the above query we are showing
city wise salary using the group
by clause.

sql> SELECT city, min(basic) FROM
employee

GROUP BY city;

in the above query we are showing
city wise minimum salary using
the group by clause.

sql> SELECT city, avg(basic) FROM
employee

GROUP BY city;

In the above query we will be showing city wise average salary.


```
sql>SELECT city, avg(basic)
      max(basic), min(basic),
sum(basic)
      FROM employee
      GROUP BY city;
```

The above query will print city wise sum, max, min & average salary.


```
sql> select city, count(basic)
      from employee
      group by city;
```

the above query will print the number of employees in each city.using count function.

```
sql> select desig, count(desig)
      from employee
      group by desig;
```

the above query will print the number of employees in each designation.

The HAVING Clause

The HAVING clause was added to SQL because the WHERE keyword could not be used with aggregate functions.

The HAVING clause enables you to specify conditions that filter which group results appear in the

final results.

The WHERE clause places conditions on the selected columns, where as the HAVING clause places conditions on groups created by the GROUP BY clause.

Syntax:

The following is the position of the HAVING clause in a query:

```
SELECT  
FROM  
WHERE  
GROUP BY  
HAVING  
ORDER BY
```

The HAVING clause must follow the GROUP BY clause in a query and must also precede the ORDER BY clause if used.

The following is the syntax of the SELECT statement, including the HAVING clause

```
sql> SELECT Desig, count(desig)
FROM employee
GROUP BY Desig
HAVING COUNT(Desig) >= 2;
```

The above query will display those designation that appears more than twice in the table and for that we are using the group by and having clause and the aggregate function.

waq) where you will print all those city where their are 2 or less than 2 salespeople;

```
sql> select city, count(city)
```

```
from salespeople
  group by city
  having count(city) <=2
  order by city desc;
```

```
-----
-----
-----
-----
-----
```

Views

Views are logical table based on real table which are called base tables.

In views our query is stored which can be executed from time to time.

You can add records, modify records through views.

Write a view called london to see

salespeople who residing in
london.

```
sql> create view london
      as select * from
salespeople where city =
'London';
```

```
=====
=====
```

```
sql> select * from london;
```

The above query will display thos
salespeople who stay in london.

```
-----
-----
```

To add records through view
london

```
sql> insert into london values
      (1900, 'James Bond',
'New York', null);
```

```
sql> insert into london values
      (1450, 'Jack Patel',
       'London', 654.34);
```

```
-----
-----
```

To see all the london records
through view

```
sql> select * from london;
```

```
-----
-----
```

To drop a view london

```
sql> drop view london
```

```
-----
-----
-----
```

Delete : is used to delete all
records.

```
*****
```

waq to delete all records in
orders table.

```
sql> delete from orders;
```

```
sql>select * from orders;
```

```
-----  
-----  
-----
```

waq to delete all salespeople of
london city;

```
sql> delete from salespeople  
        where city = 'London';
```

waq to remove details of salesman
no 1004;

```
sql> delete from salespeople  
        where snum =1004;
```

```
-----  
-----
```

Update

Update is use to modify the records or tuples provided you have permission.

waq to update all records where commission is increased by 200 rupees for all employees

```
sql>update salespeople  
      set comm = comm +200;
```

```
sql> update salespeople  
      set comm = comm -100  
      where city = 'London';
```

```
sql> update customers  
      set city = 'New York',  
Name = 'Rama'  
      where cnum = 2009;
```
